Trabalho sobre Estudo da Eficiência e Algoritmos de Ataque ao RSA

Kayky Moreira Praxedes

Abril 2026

1 Declaração de uso de IA

O uso de inteligência artificial se limitou à **correção ortográfica** e **verificação dos cálculos** realizados.

Os **conceitos citados** foram extraídos diretamente das fontes pelo autor, bem como as **geração das figuras**, **formatação do texto**, **organização dos cálculos** e **desenvolvimento lógico**.

2 Introdução

2.1 O que é criptografia:

A criptografia, do grego *cryptos* (secreto), *graphein* (códigos), pode ser entendida como o **estudo dos métodos para codificar uma mensagem** (ferramentas para garantir que apenas o alvo da mensagem tenha conhecimento de seu conteúdo secreto) [1].

2.2 Criptografia simétrica:

O primeiro modelo de criptografia que se tem conhecimento é a **Cifra de César**. Trata-se de um algoritmo de reordenação de caracteres, deslocando-os para frente ou para trás uma quantidade fixa de vezes (cifra) de maneira cíclica [1].

Figura 1: Disco de decodificação de cifra de César

Fonte: Elaborado pelo autor (2026).

Esse e outros algoritmos pertencem à classe de **sistemas criptográficos simétricos**: modelos que são caracterizados por **uma mesma chave ser utilizada para cifrar e para decifrar a mensagem** [1].

Esse paradigma não cumpre os requisitos de segurança e usabilidade para a grande maioria das demandas atuais, tendo em vista características tais como:

- **Facilidade de quebra:** Em muitos casos, métodos simples podem ser usados para descobrir seu segredo, como **análise de frequência de letras** [01], ou mesmo utilização de ***brute-force*** (tentativa e erro de maneira sistemática) [1] [2].
- **Complexidade de codificação e decodificação:** Para compensar sua falta de segurança inicial, muitas vezes são aplicados algoritmos extremamente complexos e demorados, o que tornam sua utilização lenta e praticamente impraticável [1].

- **Chave insegura:** Como a chave de codificação e decodificação são as mesmas, para a troca de informações, ambos os lados teriam de ter as chaves (pouco seguro no ambiente digital, tendo em vista a existência de *spywares*) [1].
- **Algoritmos irrestritos:** A simples descoberta do método de cifragem já configura uma falha de segurança catastrófica, como ocorre na codificação por substituição determinística [1] [3].

2.3 Criptografia assimétrica:

Durante a segunda metade do século XX, métodos mais avançados, explorando conceitos matemáticos complexos resultaram em uma revolução conceitual com o advento da **criptografia assimétrica** ou de **chave pública**, como a **criptografia RSA** (1977), **troca de chaves de Diffie-Hellman** (1976), **criptografia de curvas elípticas** (1985), etc. [1] [4] [5]

Esses métodos tiram proveito da utilização de uma **chave pública** (que todos podem ter conhecimento), a qual é **responsável pela codificação da mensagem**, enquanto uma **chave privada** (normalmente apenas uma das partes tem de ter seu conhecimento) **para decodificação** [1] [2].

Ambas as operações são realizadas de forma relativamente rápida, entretanto, sem o conhecimento da chave privada, o processo de decodificação, para não dizer impossível, é praticamente impraticável, mesmo com as ferramentas computacionais mais refinadas [1].

2.4 Objetivo:

Com esse trabalho, eu pretendo expor a **base matemática** e **provar a eficácia do modelo de criptografia assimétrica RSA** no que diz respeito à sua segurança, através de exemplos de **implementação reais** e expondo detalhes cruciais e falhas comuns de desenvolvimento.

3 Desenvolvimento

3.1 Funcionamento da criptografia RSA:

A criptografia RSA opera em três etapas distintas: **pré-codificação**, **codificação** e **decodificação** [1] [2].

1. **Pré-codificação:**

   i. **Escolha de chaves:** São definidos **2 primos** p e q que **irão compor a chave pública** n (em implementações reais esses primos possuem na casa de 100 algarismos, n possuindo pelo menos 200 algarismos) e um expoente público e (normalmente um primo pequeno, como 65.537).

   A partir desses números, é calculado um **expoente privado** d, obtido através da resolução da equação modular: $e \cdot d \equiv 1 \pmod{(p-1)(q-1)}$.

   As únicas regras para o funcionamento do algoritmo são: $p \neq q$ e $(p-1)(q-1) \not\equiv 0 \pmod{e}$ [1] [2].

   ii. **Preparação da mensagem:** Como a abordagem é numérica, é necessário transformar de alguma maneira os **caracteres da mensagem em números**. A representação numérica tem de ser suficientemente abrangente para ser uniforme (**não haver ambiguidade**). Para uma abordagem simples, transformar a letra em um número de dois dígitos é suficiente [02].

   Uma vez essa parte concluída, o número resultante é separado em blocos b. Cada bloco sequencial recebe os dígitos do número gerado no passo anterior (da esquerda para a direita), ficando com o maior valor onde $b < n$ e o algarismo mais significativo do próximo bloco não seja 0 [1].

2. **Codificação:** É realizada uma operação de codificação em cada bloco: $\mathbf{C}(b) \equiv b^e \equiv a \pmod{n}$ (a é o valor codificado do bloco $1 \leq a < n$).

3. **Decodificação:** É realizada a operação de decodificação de cada bloco resultante: $\mathbf{D}(a) \equiv a^d \equiv b \pmod{n}$ (obtém-se o mesmo valor de bloco novamente).

Após esse processo, basta transformar os blocos em caracteres novamente.

3.2 Fundamentação matemática do RSA:

O que caracteriza o funcionamento desse modelo, uma vez os blocos separados, é que o resultado de $\mathbf{D}(\mathbf{C}(b))$ invariavelmente sempre tem que ser igual à b. Para provar esse funcionamento é necessário primeiro explicitar alguns fundamentos da **Teoria dos Números**.

Conceitos:

- **Equação Diofantina e inverso modular:** Se x e y são primos entre si ($\text{mdc}(x, y) = 1$), certamente existirá um conjunto de valores: $x \cdot x' + k \cdot y = 1 \Rightarrow x \cdot x' \equiv 1 \pmod{y}$ (x' é o inverso modular de x, $1 \leq x' < y$) [1] [2].
- **Pequeno teorema de Fermat:** Se ($\text{mdc}(x, y) = 1$), $x^{y-1} \equiv 1 \pmod{y}$ [1] [2].
- **Função Totiente de Euler ($\varphi(n)$):** Para um primo qualquer p [1] [2]:
  i. A quantidade de números primos entre 1 e x, é $x - 1 \Rightarrow \varphi(x) = x - 1$.
  ii. $\varphi(x^k) = x^{k-1}(x - 1) = x^k - x^{k-1}$.
  iii. Se ($\text{mdc}(x, y) = 1$), $\varphi(x \cdot y) = \varphi(x) \cdot \varphi(y)$.
- **Teorema de Euler:** Junção do pequeno teorema de Fermat com a função Totiente de Euler: Se ($\text{mdc}(x, y) = 1$), $x^{\varphi(y)} \equiv 1 \pmod{y}$ [2].
- **Algoritmo chinês do resto:** $x \equiv y \pmod{z}$ e $x \equiv y \pmod{w} \Rightarrow x \equiv y \pmod{z \cdot w}$ [03] [1].

Dedução:

$$\mathbf{D}(\mathbf{C}(b)) \equiv \mathbf{D}(b^e) \equiv (b^e)^d \equiv b \pmod{n} \Rightarrow b^{e \cdot d} \equiv b \pmod{n}$$

Como $e \cdot d \equiv 1 \pmod{(p-1)(q-1)}$, $e \cdot d$ pode ser reescrito como: $k(p-1)(q-1) + 1,\ k \in \mathbb{Z}$. Logo:

$$b^{e \cdot d} \equiv b^{k(p-1)(q-1)+1} \equiv b \cdot (b^{(p-1)(q-1)})^k \equiv b \pmod{n}$$

Como p e q são primos, existem as seguintes possibilidades:

1. ($\text{mdc}(n, b) = 1$). Pelo teorema de Euler: $b^{(p-1)(q-1)} \equiv b^{\varphi(p)\varphi(q)} \equiv b^{\varphi(n)} \equiv 1 \pmod{n}$. Dessa maneira:

$$b \cdot (b^{(p-1)(q-1)})^k \equiv b \cdot 1 \equiv b \pmod{n} \Rightarrow b^{e \cdot d} \equiv b \pmod{n}$$

2. ($\text{mdc}(n, b) \neq 1$). Nesse caso, b é múltiplo de p ou q (não dos 2, caso contrário $b = n$, o que não é possível pois uma das condições é $b < n$).

   Assumindo que b é múltiplo de p (escolha arbitraria que não altera o resultado):

$$b^{e \cdot d} \equiv (k \cdot p)^{e \cdot d} \equiv k^{e \cdot d} \cdot p^{e \cdot d} \equiv k^{e \cdot d} \cdot 0 \equiv 0 \pmod{p} \Rightarrow b^{e \cdot d} \equiv 0 \equiv b \pmod{p}$$

   Para calcular em relação à q, como ($\text{mdc}(b, q) = 1$):

$$b^{e \cdot d} \equiv b \cdot (b^{(p-1)(q-1)})^k \equiv b \cdot (b^{(q-1)})^{k(p-1)} \equiv b \cdot (b^{\varphi(q)})^{k(p-1)} \equiv b \cdot 1^{k(p-1)} \equiv b \pmod{q}$$

   Como $b^{e \cdot d} \equiv b \pmod{p}$ e $b^{e \cdot d} \equiv b \pmod{q}$, pelo teorema chinês do resto: $b^{e \cdot d} \equiv b \pmod{n}$.

Tem-se então que: $\mathbf{DC}(b) \equiv b^{e \cdot d} \equiv b \pmod{n}$, para todos os caso.

Para finalizar a dedução, como:

$$\begin{cases} 1 \leq b < n \\ 1 \leq \mathbf{DC}(b) < n \\ \mathbf{DC}(b) \equiv b \pmod{n} \end{cases}$$

A única opção válida para satisfazer o sistema é a congruência: $\mathbf{DC}(b) = b$.

3.3 Código implementado:

Para análise da eficácia do modelo de criptografia, foram elaborados **testes práticos** através de códigos implementados pelo autor (Apêndice B). Os resultados obtidos com esses experimentos podem variar a depender do poder computacional de **hardware**, bem como variações de **software** e eventuais **otimizações**, mas, no geral, podem ser abstraídos.

O código é composto de módulos, com as funções principais sendo:

- **Cálculo de primos:** Modulo auxiliar com a implementação do algoritmo do **crivo de Eratóstenes**, tendo em vista a sua eficácia computacional [04].
- **Fatoração de um valor:** Também utilizando o módulo auxiliar, para um número k, a partir dos primos calculados até um limite $\sqrt{k}$ [05], são realizadas divisões pelos primos do crivo quando $k \equiv 0 \pmod{p}$ até $k = 1$. O tempo de fatoração é desprezível perto do tempo de cálculo de primos.

Figura 2: Testes envolvendo fatoração

```
Insira o valor a ser fatorado: 10
10 = 2 * 5

2 primo(s) calculado(s) em 7 microssegundo(s)
```

```
Insira o valor a ser fatorado: 1234567891011
1234567891011 = 3 * 7 * 13 * 67 * 107 * 630803

183855 primo(s) calculado(s) em 234 milissegundo(s)
```

```
Insira o valor a ser fatorado: 1000000000039
1000000000039 = 1000000000039 (primo)

166895 primo(s) calculado(s) em 213 milissegundo(s)
```

```
Insira o valor a ser fatorado: 1000006000009
1000006000009 = 1000003^2

166896 primo(s) calculado(s) em 216 milissegundo(s)
```

Fonte: Elaborado pelo autor (2026).

- **Cálculo do primeiro primo de n dígitos:** Utilizando o módulo auxiliar de calcular primos, uma função do código foi desenvolvida para gera o menor valor de primo com k caracteres (utilizando-se da **conjectura de Legendre**: $n^2 < primo < (n+1)^2$ [6]. Trata-se de um valor conveniente para se obter o piso (limite inferior) do tempo necessário para se fatorar um valor de n dígitos composto de apenas dois primos.

Figura 3: Testes envolvendo cálculo de primos de n dígitos

```
Insira o número de dígitos do primo: 0
Valor inválido! Não existem números com 0 dígitos!
```

```
Insira o número de dígitos do primo: 1
O menor primo com 1 dígito(s) é 2
```

```
Insira o número de dígitos do primo: 2
O menor primo com 2 dígito(s) é 11
```

```
Insira o número de dígitos do primo: 15
O menor primo com 15 dígito(s) é 100000000000031
```

Fonte: Elaborado pelo autor (2026).

- **Criptografia RSA:** Módulo completo onde é possível fazer a **codificação**, **decodificação** de uma mensagem, bem como o **teste de chaves**.

Figura 4: Testes do módulo de criptografia RSA

```
Insira as chaves (p q e): 13 17 3
Conjunto de chaves inválido! ((p - 1) * (q - 1)) % e == 0
```

```
Insira as chaves (p q e): 100000000003 100000007 5
Chaves públicas (n e): 10000000700300000021 5
Chaves privadas (n d): 10000000700300000021 4000000240080000005
```

```
Insira as chaves para codificação (n e): 10000000700300000021 5
Insira a mensagem: MAMA MIA MARCHELLO
Mensagem codificada: 7193884813498087947 5882983198923909705
```

```
Insira as chaves para decodificação (n d): 10000000700300000021 4000000240080000004
Insira a mensagem: 7193884813498087947 5882983198923909705
Mensagem decodificada: zzzMzzzzzzzUQzYzzGZ
```

```
Insira as chaves para decodificação (n d): 10000000700300000021 4000000240080000005
Insira a mensagem: 7193884813498087947 5882983198923909705
Mensagem decodificada: MAMA MIA MARCHELLO
```

Fonte: Elaborado pelo autor (2026).

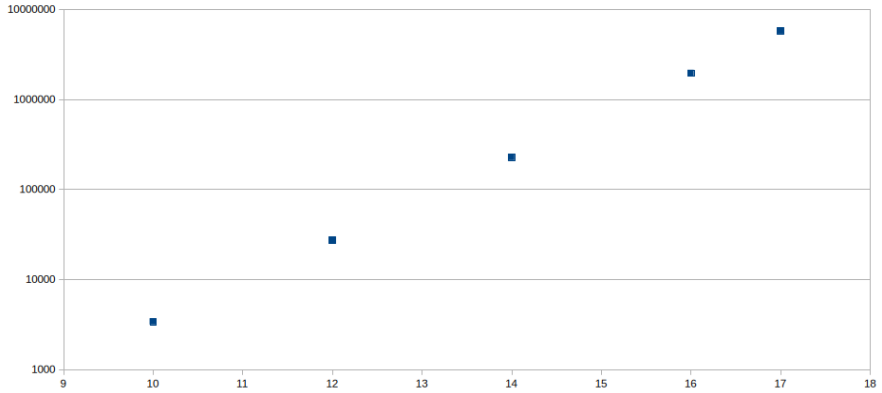
3.4 Eficácia prática do RSA:

Testando o tempo médio de fatoração de valores entre 10 à 17 dígitos [06], obtém-se:

- 10 dígitos (1000000000): 3401 primos calculados em 8.6 milissegundos.
- 12 dígitos (100000000000): 27293 primos calculados em 79 milissegundos.
- 14 dígitos (10000000000000): 227647 primos calculados em 813.2 milissegundos.
- 16 dígitos (1000000000000000): 1951957 primos calculados em 8551.8 milissegundos.
- 17 dígitos (10000000000000000): 5761455 primos calculados em 28821 milissegundos.

A partir desses dados, é possível utilizar **regressão linear** para gerar uma **função que simula o comportamento para valores maiores**, bem como **representar graficamente** essa relação.

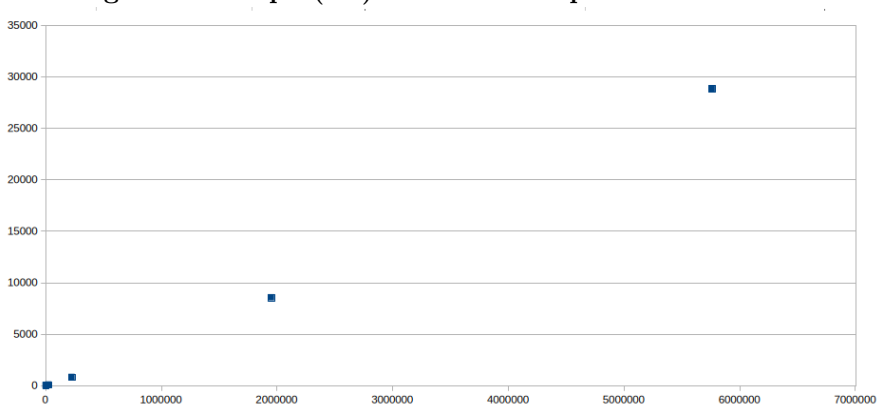
Figura 6: Número de primos calculados x Numero de dígitos - Escala logarítmica



Fonte: Elaborado pelo autor (2026).

Através do software GeoGebra, a função obtida que descreve aproximadamente o número de primos calculados em relação à quantidade de dígitos do número fatorado é: $N(n) = 0.0804722585862 \cdot 2.894414622463^n$.

Figura 7: Tempo (ms) x Número de primos calculados



Fonte: Elaborado pelo autor (2026).

Com a mesma ferramenta, a função que descreve a relação do tempo decorrido em relação ao número de primos calculados nessa máquina obtido é: $T(n) = 0.0050070989023 \cdot n - 328.3521406114272$.

Deduzindo o número de primos por milissegundo como: $\dfrac{n}{T(n)} = \dfrac{n}{0.0050070989023 \cdot n - 328.3521406114272}$. Essa função possui uma assíntota horizontal em $T = 199.716446490472 \approx 200$. Isso significa que, assintoticamente, esse computador de teste calcula 200 primos por milissegundo.

Extrapolação:

Imaginando um supercomputador capaz de calcular 1000000000 vezes mais rápido (ou seja, calculando $2 \cdot 10^{11}$ primos por milissegundo assintoticamente), para fatorar um valor n de 200 dígitos (caso real de chave pública na criptografia RSA), ele teria de calcular $N(200) \approx 1,6512 \cdot 10^{91}$ primos. $\dfrac{1,6512 \cdot 10^{91}}{2 \cdot 10^{11}} \approx 8 \cdot 10^{79}$ milissegundos (aproximadamente $2,6 \cdot 10^{69}$ anos).

O método é completamente seguro para algoritmos implementáveis em computadores comerciais, mesmo com capacidade bruta descomunalmente exagerada.

3.5 Algoritmos modernos:

Algoritmos modernos que envolvem testes sérios de quebra de chaves muito grandes (mais de 100 dígitos) derivam do **algoritmo GNFS (General Number Field Sieve)** [07].

Durante sua execução são realizadas **operações polinomiais**, **matriciais** e de **raízes quadradas**, tudo isso com números muito grandes. Para computadores comuns, esse tipo de operação causaria um *overhead* gigantesco, tendo em vista a necessidade de ***Terabytes* de memória RAM**, bem como **inúmeros processadores e placas de vídeo otimizados especialmente para esse tipo de operação matemática** (em um computador normal, se comportaria de maneira equivalente ou até pior do que o crivo) [07] [08].

Em 2020 foi gerado um novo recorde para esse algoritmo ao fatorar o maior número inteiro na criptografia RSA-250. No total, foram necessários 2.700 *core-years* (anos de execução de núcleos de computadores) [09].

4 Conclusão

Em suma, a criptografia RSA trata-se de um **modelo extremamente eficaz** no que se propõe, sendo **consideravelmente veloz na sua codificação** e **extremamente seguro**, levando em conta os padrões e ferramentas atuais, como demonstrado. Mesmo o melhor algoritmo já desenvolvido unido à um aparato computacional descomunal e colaboração coletiva direcionada, ainda sim ninguém (instituição, indivíduo ou grupo) jamais chegou perto de decodificar o modelo RSA-2048.

4.1 Análise crítica e perspectivas futuras:

Todavia, ao que tudo indica, a emergente **criptografia quântica** ameaça esse status. A principal fraqueza da criptografia RSA à esse paradigma é principalmente devido ao **algoritmo de Shor**, que permite que um computador quântico suficientemente potente fatore números inteiros grandes em **tempo polinomial** (métodos de fatoração atuais, mesmo o GNFS são subexponenciais) [10] [11].

Em teoria, um computador quântico com 1000000 de *qubits* poderia quebrar o RSA-2048 (modelo mais seguro, utilizado por governos e bancos, com números de 2048 bits ou em torno de 600 dígitos) em aproximadamente **uma semana**! Em decorrência disso, muitos sistemas estão começando uma migração para modelos **pós-quânticos** ou de **criptografia quântica** [11].

Apêndice A (informações complementares)

Introdução

[01] - As línguas possuem uma frequência de caracteres desigual, essa abordagem sendo explorada em diversos sistemas de criptografia baseados em uma modificação proporcional dos caracteres da mensagem.

Figura 8: Tabela de frequência de caracteres para a língua portuguesa

Letra	%	Letra	%	Letra	%	Letra	%
A	14,64	G	1,30	N	5,05	T	4,34
B	1,04	H	1,28	O	10,73	U	4,64
C	3,88	I	6,18	P	2,52	V	1,70
D	4,10	J	0,40	Q	1,20	X	0,21
E	12,57	L	2,78	R	6,53	Z	0,47
F	1,02	M	4,75	S	7,81		

Fonte: Criptografia - S. C. Coutinho.

Desenvolvimento

[02] - Didaticamente, no livro do Coutinho foi utilizada a conversão do alfabeto começando no número 10: A = 10, B = 11, C = 12, ..., Z = 35. No programa implementado, tendo em vista a conveniência da linguagem já utilizar caracteres como valores numéricos ordenados pela tabela ASCII (A = 65, B = 66, C = 67, ..., Z = 90), esse foi utilizado (não existe um método único).

[03] - O algoritmo chinês do resto evidenciado no livro do Coutinho na verdade diz que, dado o sistema modular:

$$\begin{cases} x \equiv y \pmod{z} \\ x \equiv k \pmod{w} \end{cases}$$

Se $(\text{mdc}(p, q) = 1)$ o sistema modular só tem uma solução x no intervalo $1 \leq x < z \cdot w$. No caso de y e k serem iguais, a solução do sistema seria $x = y = k$.

[04] - Apesar de não ser o algoritmo mais refinado para o cálculo de primos, **computacionalmente sua implementação é mais eficaz**, tendo em vista custos de alocação de memória e operação de algoritmos. Aprofundando a análise, para a fatoração de um valor n:

Métodos para geração de primos:

- **Crivo de Eratóstenes:** Utiliza **operações básicas de adição e multiplicação** (média de 1 ciclo de *clock*), bem como **armazenamento alinhado de memória** (*cache-friendly*, visto que *cache-hits* trazem o dado em 5 ciclos de *clock* em média, mas num *cache-miss*, esse número pode subir para 300), além de um crescimento basicamente linear ($\mathbf{O}(N \cdot \log\log N)$ para calcular primos até N) [1] [2] [12].
- **Crivo de Atkin:** Embora possua complexidade assintótica inferior ao crivo de Eratóstenes ($\mathbf{O}(N)$), impõe um elevado *overhead* prático ao exigir a **inversão de equações quadráticas** e o **cálculo de resíduos modulares** (até 80 ciclos de *clock*), além de acesso à memória não sequencial (muitos *cache-miss*) [12] [13].

Métodos para teste de primalidade:

- **Método de Miller-Rabin:** Algoritmo probabilístico para para teste de primalidade. É extremamente eficiente para o teste individual de números grandes ($\mathbf{O}(k \cdot \log^3 N)$), contudo, por possuir contas que envolvem **exponenciação modular** (milhares de ciclos de *clock*), sua implementação é computacionalmente pesada [12] [14].
- **O método AKS (*Agrawal-Kayal-Saxena*):** Método determinístico para teste de primalidade. Apesar de sua complexidade de tempo extremamente baixa ($\mathbf{O}(\log^{9+\epsilon} N)$, $-1.5 \leq \epsilon \leq 1.5$), o algoritmo requer a expansão polinomial de $(x + a)^n$, o que exige **alocar e manipular polinômios de grau muito alto na memória** (*overhead* nas operações exponenciais e na alocação, bem como eventuais *cache-miss*), tornando-o inviável para qualquer aplicação criptográfica real, sendo superado em desempenho até mesmo pelos métodos probabilísticos mais simples [12] [15].

Por não serem métodos de geração de primos, mas sim métodos probabilísticos de teste de primalidade, **seu uso teria de ser adaptado** (curva assintótica multiplicada pelo número de testes = $\sqrt{N}$), **piorando ainda mais seu uso para esse propósito**.

Método	**Complexidade de Tempo para essa implementação**
Crivo de Eratóstenes	$\mathbf{O}(\sqrt{N} \cdot \log\log(\sqrt{N}))$
Crivo de Atkin	$\mathbf{O}(\sqrt{N})$
Miller-Rabin (adaptado)	$\mathbf{O}(\sqrt{N} \cdot k \cdot \log^3 \sqrt{N})$
AKS (adaptado)	$\mathbf{O}(\sqrt{N} \cdot \log^{9+\epsilon} \sqrt{N})$

Fonte: Elaborado pelo autor (2026).

As fórmulas da tabela levam em conta a otimização de implementação para **calcular primos apenas até** $\sqrt{N}$ (por isso seus valores divergem das curvas descritas na explicação dos métodos).

[05] - É realizado o cálculo de fatores até $\sqrt{k}$ pois o menor fator de qualquer número composto k é menor ou igual $\sqrt{k}$ (se não achar nenhum até $\sqrt{k}$, então k é primo).

[06] - Foram realizados 5 testes para cada valor (média para consistência do valor e eliminar fatores discrepantes).

Apêndice B (implementação em C++)

***Headers*:**

```
/* tempo.h */
#include <string>
#include <chrono>
#ifndef TEMPO_H // Include guard
#define TEMPO_H
// Simplificação de nome para facilitar operações de conversão
using MICRO = std::chrono::duration<unsigned long long int, std::micro>;
// Apenas retorna uma string formatada, sem modificar nenhum dado global
std::string ajustaEscala(unsigned long long int);
#endif

/* primos.h */
#include <vector>
#include <cmath>
#include "tempo.h"
#ifndef PRIMOS_H // Include guard
#define PRIMOS_H
// Outros métodos tem de ter acesso à lista de primos
std::vector<unsigned long long int> getPrimos();
// Modifica o vetor primos e retorna o tempo que demorou
unsigned long long int calculaPrimos(const unsigned long long int &);
// Modifica o vetor primos e retorna o primo
unsigned long long int primoMin(const unsigned long long int &);
void clearPrimos();
#endif

/* fatora.h */
#include <vector>
#include <string>
#include "tempo.h"
#include "primos.h"
#ifndef FATORA_H // Include guard
#define FATORA_H
// Struct com o 2 valores long long int
struct FATOR{ // Vai ser usado como {fator, multiplicidade} e {tempo, nº primos}
    unsigned long long int num1{};
    unsigned long long int num2{};
};
// Retorna a string formatada da decomposição do valor em fatores
std::string stringFatores(const unsigned long long int &);
// Modifica um vetor de FATOR e retorna o tempo demorado para as operações
FATOR fatora(const unsigned long long int &);
#endif

/* cripto.h */
#include <string>
#include <vector>
#include <string>
#ifndef CRIPTO_H // Include guard
#define CRIPTO_H
bool testaChaves(unsigned long long int &, unsigned long long int &, unsigned long long int &);
std::string getChaves(unsigned long long int &, unsigned long long int &, unsigned long long int &);
std::string codificacao(const std::string &, unsigned long long int &, unsigned long long int &);
std::string decodificacao(const std::string &, unsigned long long int &, unsigned long long int &);
#endif
```

Módulo para o cálculo do tempo:

```
/* tempo.cpp */
#include "headers/tempo.h"
// Ajusta a escala para um formato "nnn "str"segundo(s)"
std::string ajustaEscala(unsigned long long int duracao){
    int i{};
    if(duracao > 1000){ // us - ms
        duracao /= 1000;
        i++;
    }
    std::string escala{i ? " milissegundo(s)" : " microssegundo(s)" };
    return (std::to_string(duracao) + escala);
}
```

Módulo para fatoração:

```
/* fatora.cpp */
#include "headers/fatora.h"
// Elementos privados
static std::vector<FATOR> fatores{};
// Elementos públicos
std::string stringFatores(const unsigned long long int &valor){
    std::string numeroFatorado{std::to_string(valor) + " = "};
    for(FATOR fator : fatores){
        std::string pot{fator.num2 != 1 ? ("^" + std::to_string(fator.num2)) : ""};
        numeroFatorado += (std::to_string(fator.num1) + pot);
        numeroFatorado += (fator.num1 != fatores.back().num1 ? " * " : "");
    }
    if(fatores.size() == 1 && fatores.back().num2 == 1 && (fatores.back().num1 != 1 && fatores.back().num1
    ↪  != 0)) numeroFatorado += " (primo)";
    numeroFatorado.push_back('\0');
    fatores.clear();
    return numeroFatorado;
}
FATOR fatora(const unsigned long long int &valor){
    if(valor == 0){
        fatores.push_back({0,1});
        return FATOR{0,0};
    }
    unsigned long long int copyValor{valor};
    // Output receberá tanto o tempo da fatoração quanto o número de primos calculados
    FATOR Output{calculaPrimos(valor), static_cast<unsigned long long int>(getPrimos().size())};
    for(unsigned long long int primo : getPrimos()){ // Fatora o valor
        if(primo > copyValor) break; // Fim
        while(copyValor % primo == 0){
            if(fatores.empty() || fatores.back().num1 != primo) fatores.push_back({primo, 0});
            copyValor /= primo;
            fatores.back().num2++;
        }
    } // primo || numero composto com um fator maior do que sqrt(n)
    if(fatores.empty() || copyValor != 1) fatores.push_back({copyValor, 1});
    clearPrimos();
    return Output;
}
```

Módulo para cálculo de primos:

```cpp
/* primos.cpp */
#include "headers/primos.h"
// Elementos privados
static std::vector<bool> crivo{}; // Armazena as posições válidas do Crivo
static std::vector<unsigned long long int> primos{};
// Implementação do algorítimo do "Crivo de Eratóstenes"
static void funcaoCrivo(const unsigned long long int &limite){ // Posições dos primos em lista
    crivo = std::vector<bool>(limite, true); // Define o tamanho do crivo
    crivo[0] = crivo[1] = false; // 0 e 1 não são primos mas não são compostos
    // Marca como false todos os valores compostos dos primos calculados até limite
    for(unsigned long long int i{2}; i * i < limite; i+=2){
        if(i == 2){ // Caso inicial, para permitir o crivo ser incrementado de 2 em 2 (aumentando a eficiência)
            i++; // Novo começo vai ser em i == 3, incriementando apenas números impares
        }
        if(crivo[i]) // É um primo
        for(unsigned long long int j{i * i}; j < limite; crivo[j] = false, j += i);
    }
}
// Elementos públicos
std::vector<unsigned long long int> getPrimos(){
    return primos;
}
// Calcula os primos até sqrt(valor)
unsigned long long int calculaPrimos(const unsigned long long int &valor){
    auto inicio = std::chrono::high_resolution_clock::now(); // Inicio contagem
    unsigned long long int limite{1};
    if(valor > 0) limite = static_cast<unsigned long long int>(std::sqrt(valor) + 1);
    funcaoCrivo(limite);
    if(crivo.size() >= 2){
        primos.push_back(2); // Caso inicial (se for >= 2)
        for(unsigned long long int i{3}; i < crivo.size(); i++) // Demais casos
        if(crivo[i]) primos.push_back(i);
    }
    auto fim = std::chrono::high_resolution_clock::now(); // Fim contagem
    crivo.clear();
    // Impressão do tempo
    auto duracao = std::chrono::duration_cast<MICRO>(fim - inicio);
    return duracao.count();
} // Conjectura de Legendre: n^2 < primo < (n + 1)^2
unsigned long long int primoMin(const unsigned long long int &potencia){
    // min = 10^(n - 1),  max = sucessor em potência de 10^(n - 1)
    unsigned long long int min{static_cast<unsigned long long int>(std::pow(10, potencia - 1))},
    max{static_cast<unsigned long long int>(std::pow(std::floor(std::sqrt(min)) + 1, 2))};
    calculaPrimos(max);
    for(unsigned long long int i{min + 1}; i < max; i++){
        bool ehPrimo{true};
        for(unsigned long long int primo : primos) if((i % primo || i == primo) == 0){
            ehPrimo = false;
            break;
        }
        if(ehPrimo){
            primos.clear();
            return i;
        }
    }
    return -1;
}
void clearPrimos(){
    primos.clear();
}
```

Módulo para aplicação da criptografia RSA:

```cpp
/* cripto.cpp */
#include "headers/cripto.h"
// Elementos privados
// Uso comum:
// Multiplicação binária (mantém um valor máximo de bits, pois, para as operações modulares, não é necessário o valor completo)
static unsigned long long int modMul(unsigned long long int a, unsigned long long int b, const unsigned long long int mod) {
    /* Exemplo para ilustrar a lógica em 6 bits (resultado completo teria 7 bits):
    10 == 001010b => 10 * 10 = 001010b * 001010b = (001010b * 0b) + (010100b * 1b) + (101000b * 0b) + (010000b * 1b) = 010100b + 010000b = 100100b */
    unsigned long long int result{};
    a %= mod;
    while (b > 0) {
        if (b & 1) {
            if (result >= mod - a) result -= mod; // Previne o overflow do resultado (em relação à mod)
            result += a; // 2^0 * b[0] + 2^1 * b[1] + 2^2 * b[2] + ...
        }
        if (a >= mod - a) a -= (mod - a); // Previne o overflow de a (em relação à mod)
        else a += a; // a = 2 * a
        b >>= 1; // Shift para o próximo bit de b
    }
    return result;
} // Exponenciação binária (preserva os bits menores do que n a cada operação, tornando o processo mais eficiente e sem perdas)
static unsigned long long int modPow(const unsigned long long int base, unsigned long long int exp, const unsigned long long int mod) {
    /* Exemplo para ilustrar a lógica em 6 bits (resultado completo teria 17 bits):
    10 == 001010b => 10^5 = 001010b^101b = 001010b^1b * 001010b^0b * 001010b^100b = 001010b * (001010b * 001010b * 001010b * 001010b) = 001010b * 010000b = 100000b */
    unsigned long long int result{1};
    unsigned long long int b{base};
    while (exp > 0) {
        if (exp & 1)
        result = modMul(result, b, mod); // Multiplicação do resultado até agora pela potência da base
        b = modMul(b, b, mod); // Eleva a base atual ao quadrado (b -> b^2 -> b^4 -> b^16 -> ...)
        exp >>= 1; // Shift no expoente
    }
    return result;
} // Converte a mensagem em uma string de números
static std::string charToNum(const std::string &strInput, bool reverse){
    std::string strOutput{};
    if(!reverse) // String com letras para string com numero
    for(int i{}; strInput[i] != '\0'; i++)
    strOutput += std::to_string(0 + strInput[i]);
    else // String com numero para string com letras
    for(int i{}; strInput[i] != '\0'; i += 2){
        int aux{std::stoi(strInput.substr(i, 2))};
        if((aux < 65 || aux > 90) && aux != 32) aux = 122; // Falha na decodificação
        strOutput.push_back(static_cast<char>(aux));
    }
    strOutput.push_back('\0');
    return strOutput;
} // Codificação:
// Calcula o valor de d, tal que: e * d % phi == 1
static unsigned long long int modInverse(const unsigned long long int &e, const unsigned long long int &phi) {
    /* 1) Pela definição: e * d % phi == 1 => e * d = 1 + k * phi => e * d - k * phi = 1
    Definindo a = -k: d * e + a * phi = 1
    2) É possivel transformar essa expressão em uma representação genérica: t0 * e + t1 * phi = r.
    Os valores básicos obtidos dessa relação são:
    - Para t0 = 1 e t1 = 0: r0 = t0 * phi + t1 * e = 1 * phi + 0 * e = phi
    - Para t0 = 0 e t1 = 1: r1 = t0 * phi + t1 * e = 0 * phi + 1 * e = e
    3) Através de combinações lineares, é possível obter valores menores de resto, até onde rn = 1 (caso esperado):
    q1 = r0/r1 (apenas a parte inteira)
    - r2 = r0 - q1 * r1 = phi - q1 * (t0 * phi + t1 * e) = (1 - q1 * t0) * phi - (q1 * t1) * e (achar restos menores até rn = 1)
    - t2 = t0 - q1 * t1 = -q1 * t1 (achar o valor que multiplica e)
```

```
    q2 = r1/r2
    - r3 = r1 - q2 * r2 = t0 * phi + t1 * e - q2 * ((1 - q1 * t0) * phi - (q1 * t1) * e) = a1 * phi + (q2 * q1
    ↪  * t1 + t1) * e
    - t3 = t1 - q2 * t2 = t1 - q2 * (-q1 * t1)  = q2 * q1 * t1 + t1
    ...
    q(n-1) = r(n-2)/r(n-1)
    rn = 1 = r(n-2) - q(n-1) * r(n-1) = a * phi + d * e
    tn = t(n-2) - q(n-1) * t(n-1) = d */
    unsigned long long int r0{phi}, r1{e}, t0{0}, t1{1};
    for(; r1 != 1; ) {
        unsigned long long q{r0/r1}, r2{r0 - q * r1}, t2{t0 - q * t1};
        r0 = r1;
        t0 = t1;
        r1 = r2;
        t1 = t2;
    }
    if(t1 > phi) t1 += phi; // overflow
    return t1;
} // Converte a string de números em blocos
static std::vector<unsigned long long int> numToBloco(const std::string &mensagem, const unsigned long long int
↪  &n){
    std::vector<unsigned long long int> blocos{};
    std::string aux{};
    for(int i{}; mensagem[i] != '\0'; i++){
        aux += mensagem[i];
        if(mensagem[i + 1] != '\0'){
            aux.push_back(mensagem[i + 1]);
            std::string nStr{std::to_string(n)};
            // Bloco < chave pública (método com prevenção de overflow de unsigned long long)
            if(aux.size() > nStr.size() || (aux.size() == nStr.size() && aux > nStr)){
                if(aux.back() == '0'){ // Garante que bloco[0] != 0
                    i--;
                    aux.pop_back();
                }
                aux.pop_back();
                blocos.push_back(std::stoull(aux));
                aux.clear();
            } else aux.pop_back(); // Apaga o elemento adicionado para teste
        } else { // Último elemento
            blocos.push_back(std::stoull(aux));
            aux.clear();
        }
    }
    return blocos;
}
static std::vector<unsigned long long int> preCodificacao(const std::string &mensagem, const unsigned long long
↪  int n){
    // Mensagem -> Numero -> Blocos
    return numToBloco(charToNum(mensagem, false), n);
}// Decodificação:
static std::vector<unsigned long long int> split(const std::string &mensagem) {
    std::vector<unsigned long long int> blocos{};
    std::string bloco{};
    // Separa a string pelos espaços
    for(int i{}; i <= mensagem.size(); i++){
        if((mensagem[i] == ' ' || mensagem[i] == '\0') && !bloco.empty()){
            // Caso o exista um espaço a mais no final, o bloco vazio causaria bugs
            blocos.push_back(std::stoull(bloco));
            bloco.clear();
        } else bloco.push_back(mensagem[i]);
    }
    return blocos;
}
// Converte os blocos numa string de números
static std::string blocoToNum(std::vector<unsigned long long int> blocos){
    std::string novoNumero{};
    for(unsigned long long int bloco : blocos) novoNumero += std::to_string(bloco);
    novoNumero.push_back('\0');
    blocos.clear();
    return novoNumero;
```

```
}
static std::string posDecodificacao(std::vector<unsigned long long int> blocos){
    // Blocos -> Números -> Mensagem
    return charToNum(blocoToNum(blocos), true);
}
// Elementos públicos
bool testaChaves(unsigned long long int &p, unsigned long long int &q, unsigned long long int &e){
    // Teste de overflow
    unsigned long long limiteULL = -1; // -1 vira 18446744073709551615 pelo complemento de dois
    if(p > limiteULL / q) return false; // p * q > limite do unsigned long long (transbordaria)
    // Teste de validade das chaves
    if((((p - 1) * (q - 1) % e) == 0) || (p == q)) return false;
    return true;
}
std::string getChaves(unsigned long long int &p, unsigned long long int &q, unsigned long long int &e){
    unsigned long long limiteULL = -1;
    if(p > limiteULL / q)
    return "Conjunto de chaves inválido! n >= 2^64 (limite do unsigned long long int)";
    if(((p - 1) * (q - 1) % e) == 0)
    return "Conjunto de chaves inválido! ((p - 1) * (q - 1)) % e == 0";
    if(p == q) return "Conjunto de chaves inválido! p == q";
    // Calcula o valor do inverso modular de e para (p - 1) * (q - 1)
    unsigned long long int fi{(p - 1) * (q - 1)}, d{modInverse(e, fi)};
    std::string Output{"Chaves públicas (n e): " +  std::to_string(p * q) + " " + std::to_string(e) + "\nChaves
    ↪  privadas (n d): " + std::to_string(p * q) + " " + std::to_string(d)};
    return Output;
}
std::string codificacao(const std::string &mensagem, unsigned long long int &n, unsigned long long int &e){
    std::string mensagemEmBlocos{};
    // Cria os blocos
    std::vector<unsigned long long int> blocos{preCodificacao(mensagem, n)};
    // Codificação: C(b) = (b^e) % n
    for(unsigned long long int i{}; i != blocos.size(); i++){
        blocos[i] = modPow(blocos[i], e, n);
        mensagemEmBlocos += (std::to_string(blocos[i]) + ' ');
    }
    mensagemEmBlocos.back() = '\0';
    return mensagemEmBlocos;
}
std::string decodificacao(const std::string &mensagemEmBlocos, unsigned long long int &n, unsigned long long
↪  int &d){
    // Transforma a string recebida em blocos
    std::vector<unsigned long long int> blocos{split(mensagemEmBlocos)};
    // Imprime blocos decodificados (D(a) = (a^d) % n == (b^(e * d)) % n)
    for(unsigned long long int i{}; i != blocos.size(); i++) blocos[i] = modPow(blocos[i], d, n);
    return posDecodificacao(blocos);
}
```

Módulo de testes (`main`):

```cpp
/* main.cpp */
/*
Desenvolvedor:
Kayky Moreira Praxedes
*/
#include <iostream>
#include "headers/fatora.h"
#include "headers/primos.h"
#include "headers/cripto.h"
void imprimeFatoracao(){
    std::cout << "\n===================== Fatoração =====================\n\n";
    unsigned long long int valor{};
    std::cout << "Insira o valor a ser fatorado: ";
    std::cin >> valor;
    FATOR tempoEprimos{fatora(valor)};
    std::cout << stringFatores(valor) << "\n\n" << tempoEprimos.num2 << " primo(s) calculado(s) em " <<
    ↪ ajustaEscala(tempoEprimos.num1) << "\n\n";
}
void imprimeMinPrimo(){
    std::cout << "\n================= Cálculo do primo ==================\n\n";
    int valor{};
    std::cout << "Insira o número de dígitos do primo: ";
    std::cin >> valor;
    if(valor > 0) std::cout << "O menor primo com " << valor << " dígito(s) é " << primoMin(valor) << "\n\n";
    else std::cout << "Valor inválido! Não existem números com 0 dígitos!\n\n";
}
void criptografia(){
    std::cout << "\n=================== Criptografia RSA =================\n\n";
    short int opcao{};
    while(true){
        unsigned long long int p{}, q{}, e{}, n{}, d{};
        std::string mensagem{};
        std::cout << "1) Verificar um conjunto de chaves\n2) Codificar uma mensagem\n3) Decodificar uma
        ↪ mensagem\n0) Encerrar\nOpção: ";
        std::cin >> opcao;
        switch (opcao){
            case 1:
            std::cout << "\nInsira as chaves (p q e): ";
            std::cin >> p >> q >> e;
            std::cout << getChaves(p, q, e) << "\n\n";
            break;
            case 2:
            std::cout << "\nInsira as chaves para codificação (n e): ";
            std::cin >> n >> e;
            std::cin.ignore(); // Limpa o buffer
            std::cout << "Insira a mensagem: ";
            std::getline(std::cin, mensagem);
            std::cout << "Mensagem codificada: " << codificacao(mensagem, n, e) << "\n\n";
            break;
            case 3:
            std::cout << "\nInsira as chaves para decodificação (n d): ";
            std::cin >> n >> d;
            std::cin.ignore(); // Limpa o buffer
            std::cout << "Insira a mensagem: ";
            std::getline(std::cin, mensagem);
            std::cout << "Mensagem decodificada: " << decodificacao(mensagem, n, d) << "\n\n";
            break;
            default:
            std::cout << std::endl;
            return;
        }
    }
}
int main(){
    // Menu
    short int opcao{};
    while(true){
        std::cout << "\n~~~~~~~~~~~~~~~~~~~ Menu principal ~~~~~~~~~~~~~~~~~~~\n\n";
```

```
        std::cout << "1) Fatorar um número\n2) Calcular o menor primo com n dígitos\n3) Codificar e decodificar
        ↪  uma mensagem usando RSA\n0) Encerrar\nOpção: ";
        std::cin >> opcao;
        switch (opcao){
            case 1:
            imprimeFatoracao();
            break;
            case 2:
            imprimeMinPrimo();
            break;
            case 3:
            criptografia();
            break;
            case 0:
            std::cout << "\nAplicação encerrada!" << std::endl;
            return 0;
            default:
            std::cout << "\nOpção inválida!\n\n";
        }
    }
}
```

Fontes

1. COUTINHO, S. C. Criptografia. 1. ed. Rio de Janeiro: IMPA, 2016.
2. KOBLITZ, Neal. A Course in number theory and cryptography. 2. ed. New York: Springer, 1994.
3. STALLINGS, William. Cryptography and network security: principles and practice. 7. ed. Boston: Pearson, 2017.
4. Troca de chaves de Diffie–Hellman. [Wikipédia]. Acesso em: 13 abr. 2026.
5. Criptografia de curva elíptica. [Wikipédia]. Acesso em: 13 abr. 2026.
6. Legendre's conjecture. [Wikipédia]. Acesso em: 13 abr. 2026.
7. Uma rápida introdução à criptografia: Parte III. [Medium]. Acesso em: 13 abr. 2026.
8. General number field sieve. [Wikipédia]. Acesso em: 13 abr. 2026.
9. Cientistas quebram RSA-250. [Mente Binária]. Acesso em: 14 abr. 2026.
10. Safeguarding cryptocurrency by disclosing quantum vulnerabilities responsibly. [Google Research]. Acesso em: 14 abr. 2026.
11. How quantum computing affects cryptography. [The Quantum Insider]. Acesso em: 14 abr. 2026.
12. PATTERSON, David A.; HENNESSY, John L. Computer architecture: a quantitative approach. 5. ed. Waltham: Morgan Kaufmann, 2012
13. Algoritmo mais rápido para encontrar números primos. [Baeldung]. Acesso em: 14 abr. 2026.
14. Teoria dos números - testes de primalidade. [Applied Cryptography Group | Stanford]. Acesso em: 14 abr. 2026.
15. O teste de primalidade AKS. [Terence Tao | WordPress]. Acesso em: 14 abr. 2026.
16. DANTAS, Divane Aparecida de Moraes; RIBEIRO, Frederico Augusto Menezes. Disciplina: Tópicos especiais em matemática: Criptografia. Curso de graduação em Engenharia de Computação – Centro Federal de Educação Tecnológica de Minas Gerais (CEFET-MG), 2026.